Index 1: Tạo và quản lý Index

Khái niệm, DDL, metadata, EXPLAIN và quy trình thiết kế

Bài giảng Database Management Systems

MySQL 8.x | SQL Indexing & Query Optimization

Tháng 11 năm 2026

Lộ trình và mục tiêu

1. Động cơ và nền tảng
2. Chuẩn bị dữ liệu và tạo index
3. Đọc metadata và EXPLAIN
4. Dọn dẹp và thiết kế theo workload

Nội dung phần học

1. Động cơ và nền tảng

2. Chuẩn bị dữ liệu và tạo index

3. Đọc metadata và EXPLAIN

4. Dọn dẹp và thiết kế theo workload

Sau bài học, sinh viên làm được gì?

- Giải thích vì sao index có thể giảm số row cần đọc và khi nào lợi ích không đáng kể.
- Tạo single-column và composite index bằng `CREATE INDEX` / `ALTER TABLE`.
- Xem index qua `SHOW INDEX`, `SHOW CREATE TABLE`, `INFORMATION_SCHEMA.STATISTICS`.
- Đọc các trường cơ bản trong `EXPLAIN` và kiểm tra giả thuyết thiết kế.
- Xóa index thử nghiệm an toàn, đánh giá trade-off read/write/storage.

Vấn đề: truy vấn tìm khách hàng

Giả sử bảng có 1 triệu khách hàng; ứng dụng thường xuyên tìm theo quốc gia.

- **Table scan:** có thể phải đọc nhiều row để kiểm tra `country`.
- **Index lookup:** tìm dải khóa `country = USA`, sau đó truy cập các row liên quan.
- Optimizer chọn cách có chi phí ước lượng thấp hơn, không phải lúc nào cũng chọn index.

```sql
SELECT customer_id, first_name, last_name
FROM customer_search
WHERE country = 'USA';
```

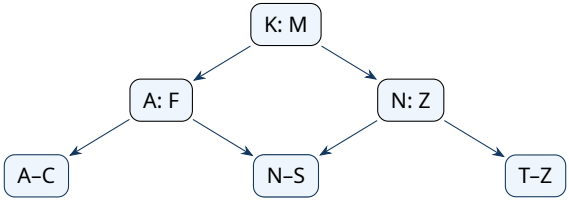
Điểm cần nhớ

Index là đường truy cập dữ liệu, không phải bản sao sắp xếp của toàn bộ table theo mọi trường.

Trực giác B-tree index

B-tree tổ chức key theo thứ tự để hỗ trợ tra cứu/range scan. Hình sau chỉ minh họa ý tưởng, không mô phỏng các page InnoDB.

- Equality lookup nhắm tới một vùng khóa; range lookup duyệt các key liên tiếp.
- Index vẫn có chi phí duy trì khi thêm/sửa/xóa dữ liệu.



Các toán tử thường hưởng lợi từ B-tree

Predicate	Khả năng khai thác index
`country = ?`	Equality lookup
`credit_limit >= ?`	Range scan
`order_date BETWEEN ? AND ?`	Range scan
`email LIKE 'abc%'`	Prefix/range search
`email LIKE '%abc'`	B-tree thường không hỗ trợ tìm đầu chuỗi

Điểm cần nhớ

Đây là khả năng kỹ thuật, không bảo đảm optimizer sẽ chọn index trong mọi dataset.

Đánh đổi: đọc nhanh hơn không miễn phí

Hoạt động	Tác động khi có nhiều secondary index
`SELECT` có điều kiện	Có thể giảm I/O, hỗ trợ join và sort
`INSERT`	Thêm entry vào từng index liên quan
`UPDATE` key	Sửa entry cũ/mới trong index
`DELETE`	Loại entry khỏi index
Storage / Cache	Tốn disk, buffer pool và tài nguyên quản trị

Điểm cần nhớ

Tối ưu cho workload tổng thể, không phải một truy vấn riêng lẻ.

Primary key, secondary, composite

Loại	Ví dụ	Đặc tính
Primary key	`PRIMARY KEY(id)`	InnoDB thường dùng làm clustered key
Secondary	`INDEX(country)`	Key phụ để tra cứu; chứa PK row
Composite	`INDEX(country, credit_limit)`	Thứ tự key parts quyết định mẫu truy vấn

Thận trọng

Hai index `(country)` và `(credit_limit)` không tương đương một index `(country, credit_limit)`.

Nội dung phần học

1. Động cơ và nền tảng
2. Chuẩn bị dữ liệu và tạo index
3. Đọc metadata và EXPLAIN
4. Dọn dẹp và thiết kế theo workload

Schema thực hành: index_lab

Tutorial sử dụng bản sao ba bảng từ cơ sở dữ liệu `classicmodels` (nạp dữ liệu vào schema riêng).

```
CREATE DATABASE IF NOT EXISTS index_lab;
USE index_lab;

-- Bang co trong lab
-- customer_search(customer_id PK, first_name,
--   last_name, email, country, credit_limit)
-- order_search(order_number PK, customer_id,
--   order_date, required_date, shipped_date, status)
-- product_search(product_code PK, product_name,
--   product_line, quantity_in_stock, buy_price, msrp)
```

Ghi nhớ: Tạo bảng và nạp dữ liệu từ `classicmodels` theo tutorial. Không thao tác trực tiếp trên các bảng production.

Ví dụ DDL: tạo bảng customer_search

```sql
CREATE TABLE customer_search (
  customer_id INT NOT NULL,
  first_name VARCHAR(50) NOT NULL,
  last_name VARCHAR(50) NOT NULL,
  email VARCHAR(120),
  country VARCHAR(50) NOT NULL,
  credit_limit DECIMAL(10,2) NOT NULL DEFAULT 0,
  PRIMARY KEY (customer_id)
) ENGINE = InnoDB;
```

Điểm cần nhớ

Primary key được khai báo khi tạo table; secondary index có thể tạo sau khi quan sát workload.

Ví dụ DDL: bảng order_search

```sql
CREATE TABLE order_search (
  order_number INT NOT NULL,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  required_date DATE NOT NULL,
  shipped_date DATE NULL,
  status VARCHAR(20) NOT NULL,
  PRIMARY KEY (order_number)
) ENGINE = InnoDB;
```

Điểm cần nhớ

`order_number` là PK; `customer_id` và `order_date` sẽ được xét để tạo index theo workload.

Ví dụ DDL: bảng product_search

```sql
CREATE TABLE product_search (
  product_code VARCHAR(15) NOT NULL,
  product_name VARCHAR(100) NOT NULL,
  product_line VARCHAR(50) NOT NULL,
  quantity_in_stock SMALLINT NOT NULL,
  buy_price DECIMAL(10,2) NOT NULL,
  msrp DECIMAL(10,2) NOT NULL,
  PRIMARY KEY (product_code)
) ENGINE = InnoDB;
```

Điểm cần nhớ

Khởi tạo cả ba table trong lab trước khi tạo index; lệnh đầy đủ có trong file SQL setup đính kèm.

Nạp hai bảng order và product

```
INSERT INTO order_search
(order_number, customer_id, order_date,
 required_date, shipped_date, status)
SELECT orderNumber, customerNumber, orderDate,
       requiredDate, shippedDate, status
FROM classicmodels.orders;

INSERT INTO product_search
SELECT productCode, productName, productLine,
       quantityInStock, buyPrice, MSRP
FROM classicmodels.products;
```

Điểm cần nhớ

Sau khi nạp: `SELECT COUNT(*) FROM order_search;` và tương tự với product, customer.

Sao chép dữ liệu từ classicmodels

```sql
INSERT INTO customer_search
  (customer_id, first_name, last_name,
   email, country, credit_limit)
SELECT customerNumber, contactFirstName,
       contactLastName,
       CONCAT('customer', customerNumber, '@example.test'),
       country, COALESCE(creditLimit, 0)
FROM classicmodels.customers;

SELECT COUNT(*) AS n FROM customer_search;
```

Điểm cần nhớ

Có thể dùng `COALESCE` để xử lý NULL theo schema đích. Đây là dữ liệu lab, không phải số liệu benchmark production.

Tạo single-column index

Mục tiêu: hỗ trợ lọc khách hàng theo `country`.

- Đặt tên theo dạng `idx_<feature>_<key>`.
- Dùng `EXPLAIN` để kiểm tra index có được chọn hay không.

```sql
CREATE INDEX idx_customer_country
ON customer_search (country);

SHOW INDEX FROM customer_search;

EXPLAIN
SELECT customer_id, first_name, last_name
FROM customer_search
WHERE country = 'USA';
```

Tạo index qua ALTER TABLE

```sql
ALTER TABLE product_search
ADD INDEX idx_product_line (product_line);

-- Nhanh cho truy van theo dong san pham?
EXPLAIN SELECT product_code, product_name
FROM product_search
WHERE product_line = 'Motorcycles';
```

Điểm cần nhớ

`CREATE INDEX` và `ALTER TABLE ... ADD INDEX` đều tạo secondary index; cần phân biệt với `ADD PRIMARY KEY`.

Composite index: thứ tự rất quan trọng

- `customer_id` dùng equality để xác định đoạn key.
- `order_date` dùng range/order trong đoạn `customer_id` đã cố định.
- `WHERE order_date >= ?` mà thiếu `customer_id` thường không khai thác trực tiếp leftmost prefix hiệu quả.

```
CREATE INDEX idx_order_customer_date
ON order_search (customer_id, order_date);

EXPLAIN SELECT order_number, order_date, status
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01'
ORDER BY order_date;
```

Khai báo index ngay trong CREATE TABLE

- Hữu ích khi thiết kế schema mới hoặc migration có kiểm soát.
- Không có quy tắc phải thêm index cho mọi foreign key hay mọi column; kiểm tra yêu cầu ràng buộc và query workload.

```
CREATE TABLE example_orders (
  order_id INT NOT NULL,
  customer_id INT NOT NULL,
  order_date DATE NOT NULL,
  PRIMARY KEY (order_id),
  INDEX idx_customer_date (customer_id, order_date)
) ENGINE = InnoDB;
```

Naming convention và index trùng lặp

Tên đề xuất	Nhận xét
`idx_customer_country`	Đọc được table + column
`idx_order_customer_date`	Thể hiện composite index
`uq_customer_email`	Gợi ý unique business key
`idx1` / `new_index`	Khó review và bảo trì

Điểm cần nhớ

Index `(customer_id, order_date)` có thể đã hỗ trợ lookup theo `customer_id`; cân nhắc trước khi tạo thêm `(customer_id)`.

Nội dung phần học

1. Động cơ và nền tảng

2. Chuẩn bị dữ liệu và tạo index

3. Đọc metadata và EXPLAIN

4. Dọn dẹp và thiết kế theo workload

SHOW INDEX: những cột quan trọng

```sql
SHOW INDEX FROM order_search;

SHOW INDEX FROM order_search
WHERE Key_name = 'idx_order_customer_date';
```

Field	Cách hiểu
`Key_name`	Tên index
`Seq_in_index`	Vị trí key part, 1 rồi 2…
`Column_name`	Tên cột tương ứng
`Non_unique`	0: unique; 1: cho phép trùng
`Cardinality`	Ước lượng số giá trị phân biệt

Composite index được mô tả bằng nhiều dòng

Ví dụ `idx_order_customer_date` có thể xuất hiện hai dòng trong kết quả `SHOW INDEX`.

- Một tên index không nhất thiết tương ứng một dòng metadata.
- `Seq_in_index` cho biết **thứ tự**, không chỉ tập các cột.
- Có thể thêm `Sub_part`, `Collation`, `Visible`, `Index_type` khi phân tích sâu.

Key_name	Seq_in_index	Column_name
idx_order_customer_date	1	customer_id
idx_order_customer_date	2	order_date

SHOW CREATE TABLE: kiểm tra toàn bộ DDL

- Kiểm tra engine (ví dụ InnoDB), columns, primary key, secondary index.
- Dùng sau migration để xác nhận thay đổi đã được áp dụng.
- Khác `DESCRIBE`: `SHOW CREATE TABLE` cho DDL gần với cấu trúc lưu trên server.

```
SHOW CREATE TABLE order_search;
```

Metadata report bằng INFORMATION_SCHEMA

```sql
SELECT INDEX_NAME, NON_UNIQUE, SEQ_IN_INDEX,
       COLUMN_NAME, CARDINALITY, IS_VISIBLE
FROM INFORMATION_SCHEMA.STATISTICS
WHERE TABLE_SCHEMA = 'index_lab'
  AND TABLE_NAME = 'order_search'
ORDER BY INDEX_NAME, SEQ_IN_INDEX;
```

Điểm cần nhớ

Có thể dùng SQL để audit nhiều bảng và phát hiện index không mong muốn; `CARDINALITY` là ước lượng.

EXPLAIN: xem optimizer dự định làm gì

- `possible_keys`: những index có thể được xét.
- `key`: index thực sự được chọn; `NULL` không đồng nghĩa tối ưu sai.
- `rows`: số row ước lượng sẽ xem xét; phụ thuộc statistics.

```
EXPLAIN
SELECT order_number, order_date, status
FROM order_search
WHERE customer_id = 103
  AND order_date >= '2004-01-01';
```

Đọc EXPLAIN có hệ thống

Field	Câu hỏi phân tích
`type`	Kiểu access: ALL, range, ref, const...
`possible_keys`	Optimizer có ứng viên nào?
`key`, `key_len`	Dùng index nào và chiều dài key?
`rows`, `filtered`	Ước lượng row được đọc / qua filter?
`Extra`	`Using where`, `Using index`, `Using filesort`?

Điểm cần nhớ

Không suy ra latency trực tiếp từ `rows`; đo thêm thời gian và actual rows nếu môi trường cho phép.

Ví dụ phân tích: key = NULL

Giả sử `EXPLAIN` cho bảng 100 dòng báo `type=ALL`, `key=NULL`. Đây chưa phải lỗi!

- Full scan 100 dòng đôi khi ít tốn chi phí hơn đi index rồi lấy row.
- Nếu predicate trả 70% dữ liệu, secondary index có thể không đáng giá.
- `SELECT *` làm index-only scan khó hơn vì cần nhiều cột.
- Trước khi ép index: xem số row, distribution, statistics, predicate và đo đạc.

Điểm cần nhớ

Mục tiêu không phải biến mọi `key` thành một tên index; mục tiêu là giảm chi phí thực tế.

EXPLAIN trước và sau CREATE INDEX

- Ghi nhận `possible_keys`, `key`, `rows`, `Extra` trước/sau.
- Giữ nguyên dữ liệu và query để kết luận có ý nghĩa.

```
-- 1. Baseline (truoc khi tao index)
EXPLAIN SELECT order_number, order_date
FROM order_search WHERE customer_id = 103;

-- 2. Tao index
CREATE INDEX idx_order_customer
ON order_search (customer_id);

-- 3. Chay lai EXPLAIN cua query ban dau
```

Nội dung phần học

1. Động cơ và nền tảng

2. Chuẩn bị dữ liệu và tạo index

3. Đọc metadata và EXPLAIN

4. Dọn dẹp và thiết kế theo workload

ANALYZE TABLE và statistics

- `ANALYZE TABLE` cập nhật statistics phục vụ ước lượng cost.
- Plan có thể thay đổi, nhưng **không bảo đảm** luôn nhanh hơn.
- Không chạy bảo trì không cần thiết trong giờ cao điểm nếu bảng lớn.

```
ANALYZE TABLE order_search;

EXPLAIN SELECT order_number, order_date
FROM order_search
WHERE customer_id = 103;
```

DROP INDEX và ALTER TABLE DROP INDEX

- Chỉ xóa secondary indexes mình đã tạo trong schema lab.
- Primary key dùng cú pháp và quy tắc khác; không xóa tùy tiện.

```
DROP INDEX idx_customer_country
ON customer_search;

ALTER TABLE product_search
DROP INDEX idx_product_line;

SHOW INDEX FROM customer_search;
```

Thận trọng

MySQL DDL có thể gây implicit commit; không giả định ROLLBACK sẽ khôi phục một index đã DROP.

Checklist trước khi xóa index

1. Xác nhận `SHOW INDEX` và các query đang phụ thuộc.
2. Kiểm tra index có tham gia hỗ trợ foreign key hoặc constraint không.
3. So sánh plan với và không có index trên môi trường test.
4. Cân nhắc invisible index (nếu version hỗ trợ) trước khi drop.
5. Chuẩn bị migration/rollback tạo lại index và thời điểm triển khai.

Thiết kế bắt đầu từ truy vấn

- Điều kiện equality trước giúp nhóm order theo customer.
- Key part `order_date` có thể phục vụ sắp xếp; MySQL có thể duyệt B-tree ngược.
- DESC key part đặc biệt đáng cân nhắc khi có mixed sort; học sâu trong Index 2.

```sql
SELECT * FROM order_search
WHERE customer_id = ?
ORDER BY order_date DESC
LIMIT 20;

-- Index ung vien
CREATE INDEX idx_order_customer_date
ON order_search (customer_id, order_date);
```

Bài tập có hướng dẫn: khách hàng theo quốc gia

Yêu cầu: lọc `country = USA`, điều kiện hạn mức tín dụng, rồi kiểm tra plan.

```sql
-- (1) De xuat index
CREATE INDEX idx_customer_country_credit
ON customer_search (country, credit_limit);

-- (2) Test
EXPLAIN SELECT customer_id, credit_limit
FROM customer_search
WHERE country = 'USA' AND credit_limit >= 50000;
```

Điểm cần nhớ

Tự trả lời: Nếu chỉ lọc `credit_limit`, index này có còn là lựa chọn chính không? Vì sao?

Bài tập tổng hợp: sản phẩm + sort

Query ưu tiên sản phẩm `Motorcycles` với giá MSRP cao trước.

1. Đề xuất index có thứ tự cột phù hợp.
2. Viết lệnh `CREATE INDEX` và xem `SHOW INDEX`.
3. Chạy `EXPLAIN` rồi mô tả `key`, `rows`, `Extra`.
4. Giải thích tại sao bảng nhỏ vẫn có thể scan.

```sql
SELECT product_code, product_name, msrp
FROM product_search
WHERE product_line = 'Motorcycles'
ORDER BY msrp DESC;
```

Ghi nhớ: Gợi ý: `(product_line, msrp)`; thực nghiệm không nhất thiết cho cùng một plan ở mọi server.

Lỗi thường gặp và cách sửa

Sai lầm	Cách sửa
Tạo index cho mọi cột	Dựa workload, write cost và EXPLAIN
Xem `key=NULL` là lỗi	Đo cost trên table nhỏ/low selectivity
Tạo nhiều index trùng prefix	Kiểm kê composite index hiện có
DROP trên production để thử	Dùng schema test, kế hoạch rollback
Suy luận từ thời gian < 1ms	Dùng nhiều runs và dataset đủ lớn

Bài tập cuối bài: thiết kế + quản trị

- Tạo `idx_order_customer_date(customer_id,order_date)`.
- Viết `SHOW CREATE TABLE` và query metadata `INFORMATION_SCHEMA.STATISTICS`.
- So sánh `EXPLAIN` trước/sau index cho equality + range.
- Dùng `ANALYZE TABLE`, giải thích tại sao plan có thể vẫn giữ nguyên.
- Đề xuất danh sách index lab có thể xóa và kế hoạch kiểm tra.

Tổng kết Index 1

- **Index:** hỗ trợ truy cập theo key nhưng tăng chi phí DML/storage.
- **DDL:** `CREATE INDEX`, `ALTER TABLE ADD/DROP INDEX`, `DROP INDEX`.
- **Metadata:** `SHOW INDEX`, `SHOW CREATE TABLE`, `INFORMATION_SCHEMA`.
- **Đánh giá:** `EXPLAIN` + statistics + benchmark, không chỉ nhìn tên `key`.
- **Bước tiếp:** Index 2 nghiên cứu loại index và quy tắc chọn theo workload.

Điểm cần nhớ

Tham khảo: vudmvn.github.io/dbms-course → Index Optimization → Index 1.